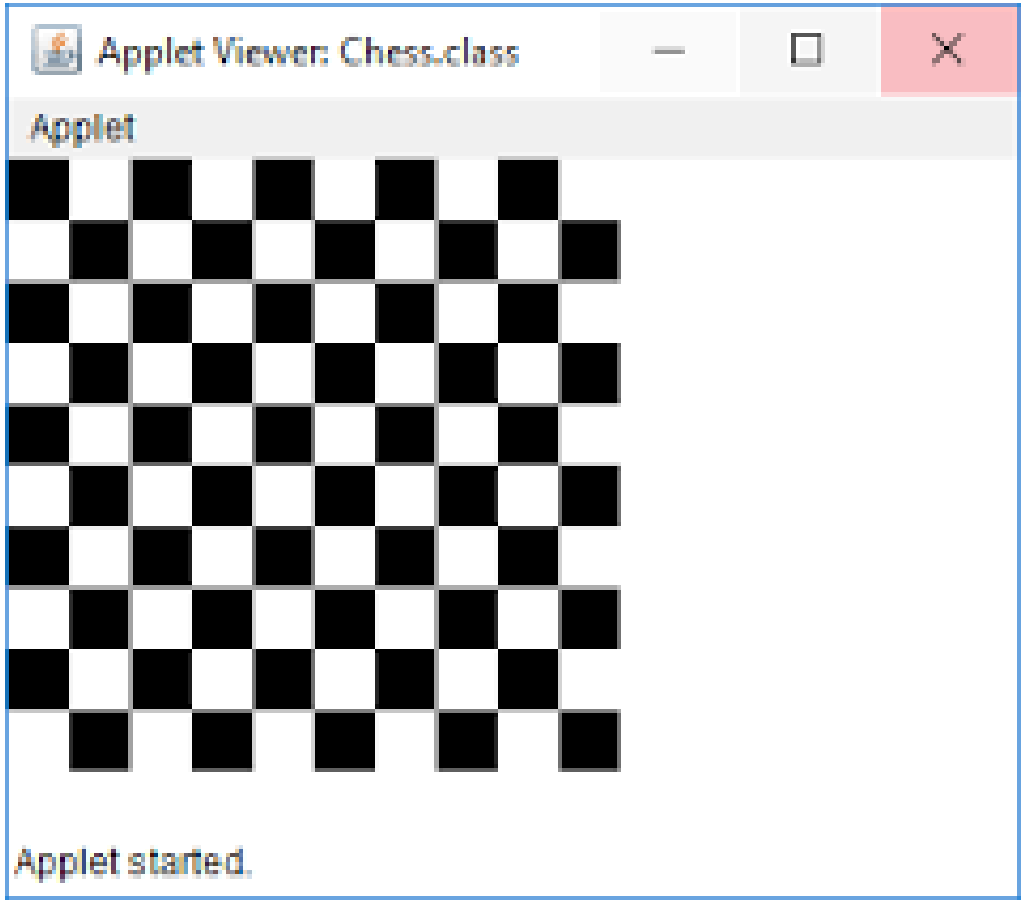
Introduction to Applet


Applet Viewer: Chess.class
Applet
Applet started.

What is Applet?

- An applet is a Java program that can be embedded into a web page.
- It runs inside the web browser and works at client side.
- An applet is embedded in an HTML page using the APPLET tag and hosted on a web server.

Important points :

1. All applets are sub-classes (either directly or indirectly) of java.applet.Applet class.
2. Applets are not stand-alone programs. Instead, they run within either a web browser or an applet viewer. JDK provides a standard applet viewer tool called applet viewer.
3. In general, execution of an applet does not begin at main() method.
4. Output of an applet window is not performed by System.out.println(). Rather it is handled with various AWT methods, such as drawString().

```
// A Hello World Applet
// Save file as HelloWorld.java

import java.applet.Applet;
import java.awt.Graphics;

// HelloWorld class extends Applet
public class HelloWorld extends Applet
{
    // Overriding paint() method
    public void paint(Graphics g)
    {
        g.drawString("Hello World", 20, 20);
    }
}
```

Module 1- Section 3

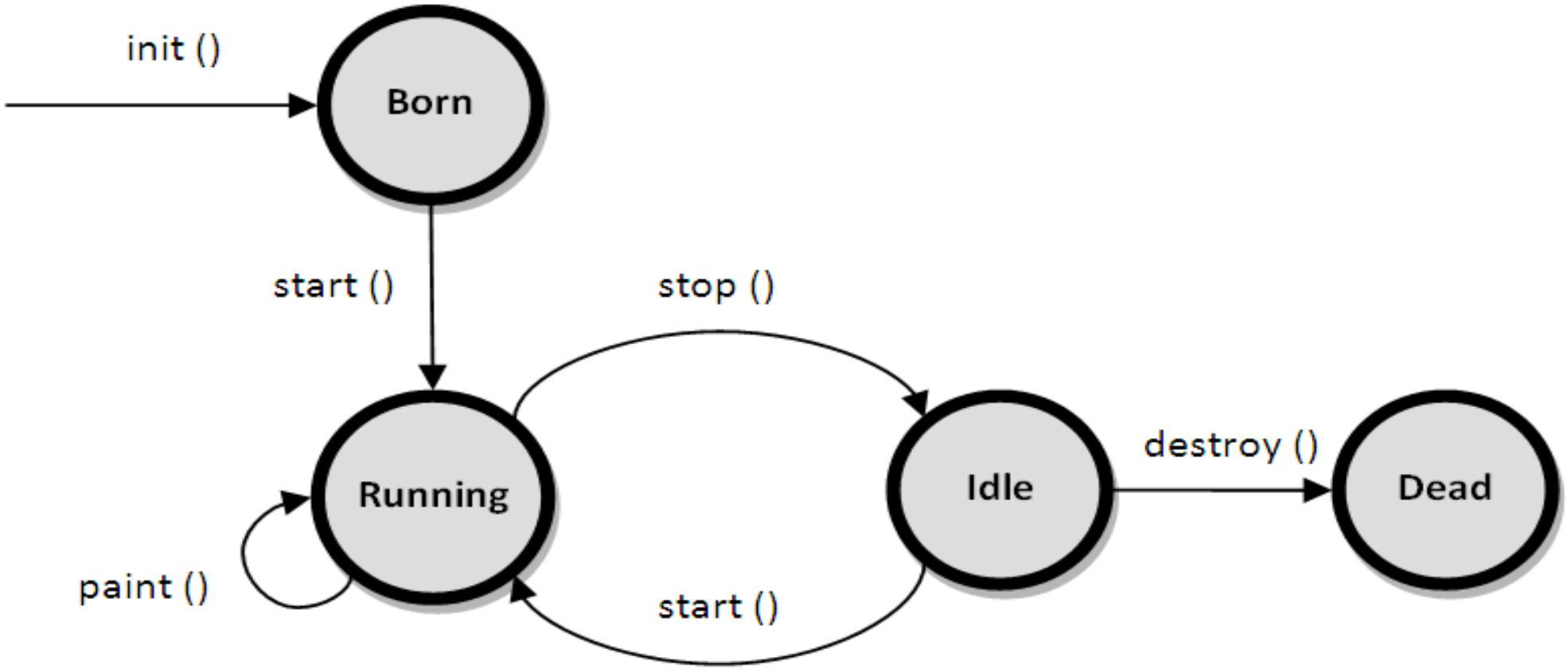
3.3 Applet Lifecycle

Applet Lifecycle

Applet Lifecycle

- The applet goes through a set of states during various phases in its life cycle.
- Applet starts its life cycle form Born state and goes through various states such as Running state, Idle state, and Dead state.

Applet Lifecycle

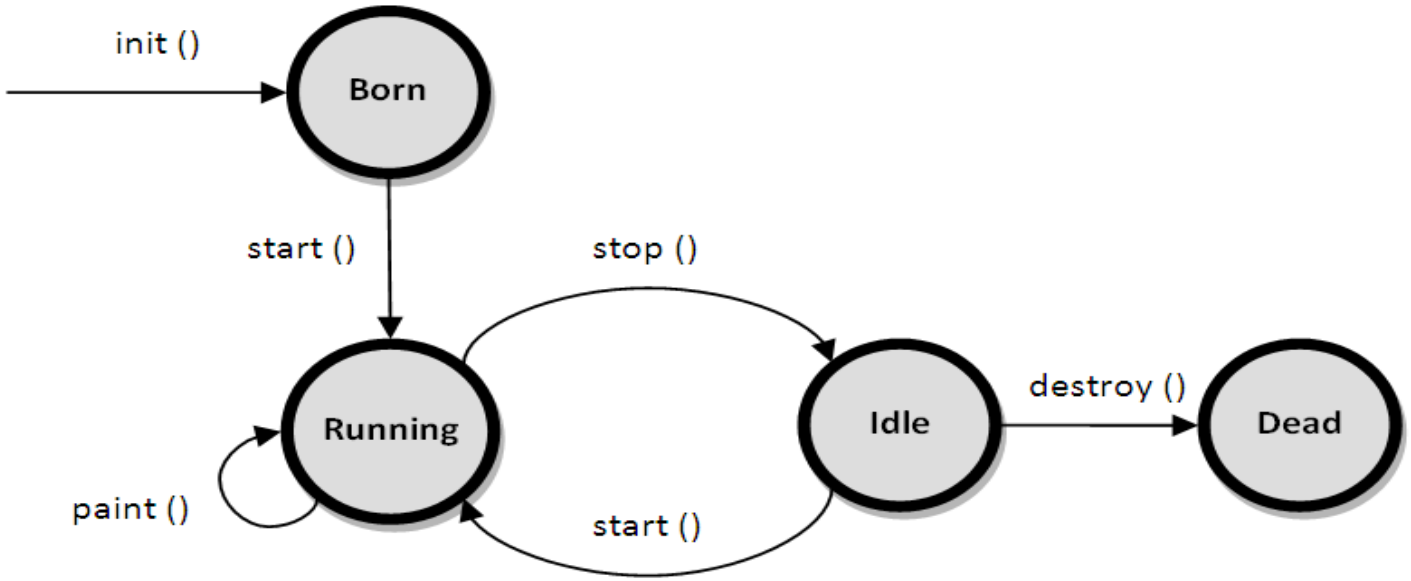


Applet Lifecycle

- The life cycle of an applet starts with *init()* method and ends with *destroy()* method. Other life cycle methods are *start(), stop()* and *paint()*.
- The methods to execute only once in the applet life cycle are *init()* and *destroy()*. Other methods execute multiple times.
- Figure is showing life cycle of applet.

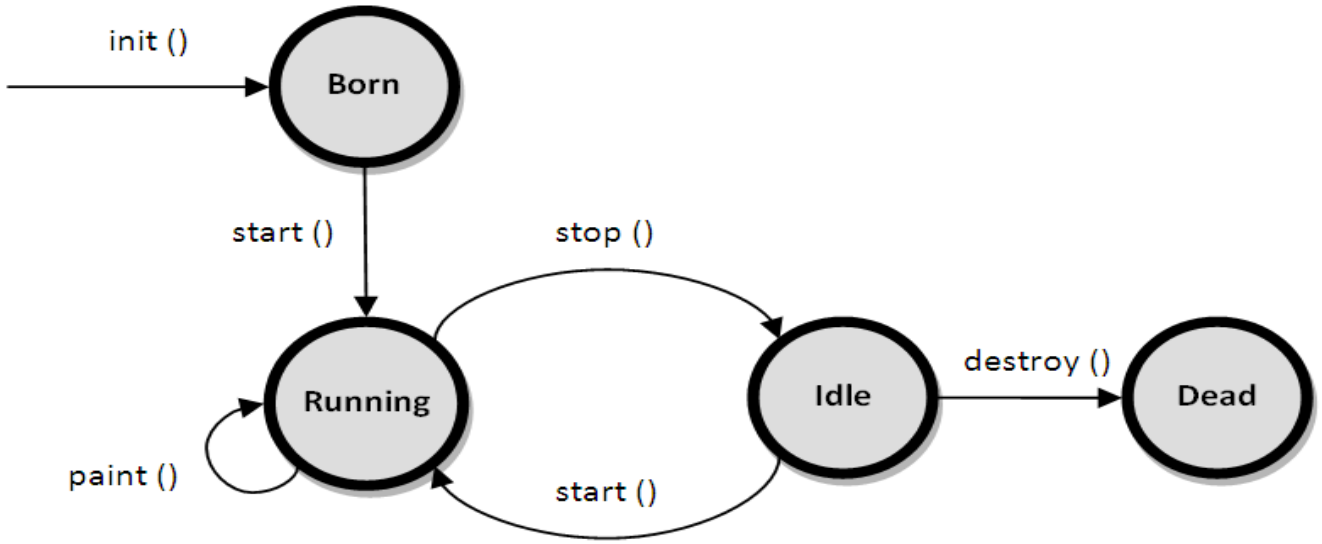
Applet Lifecycle Methods

- **init():** The init() method is the first method to execute when the applet is executed.
- Variable declaration and initialization operations are performed in this method.



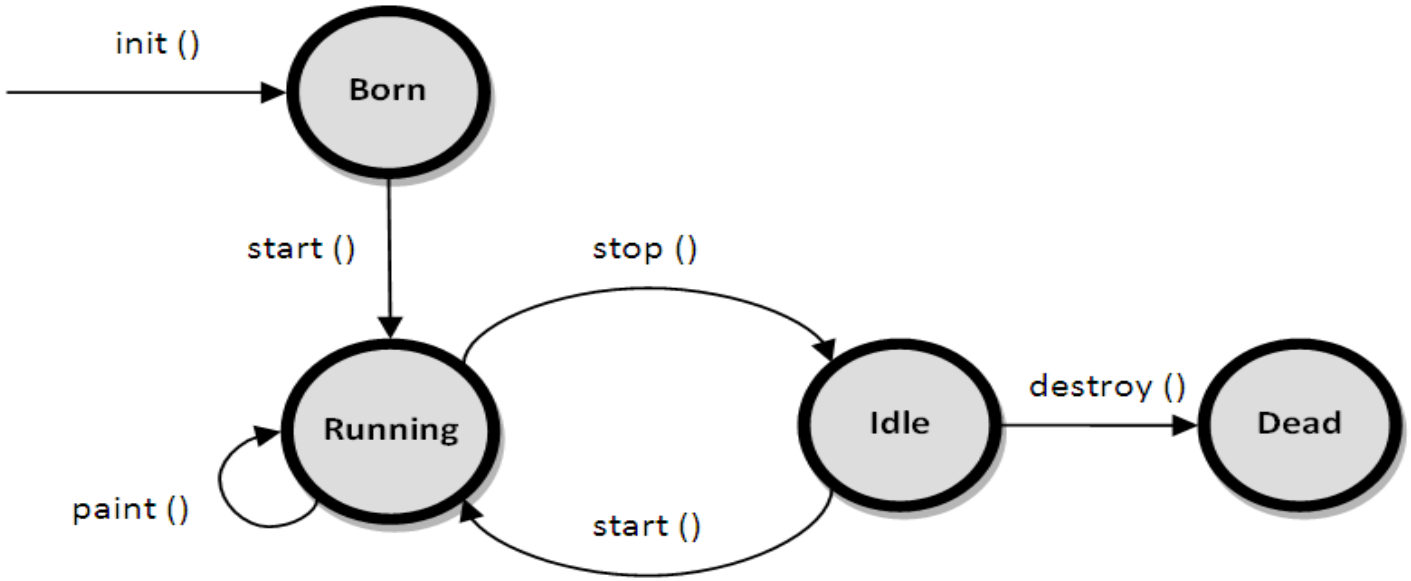
Applet Lifecycle Methods

- **start():** The start() method contains the actual code of the applet that should run.
- The start() method executes immediately after the *init()* method. It also executes whenever the applet is restored, maximized or moving from one tab to another tab in the browser.



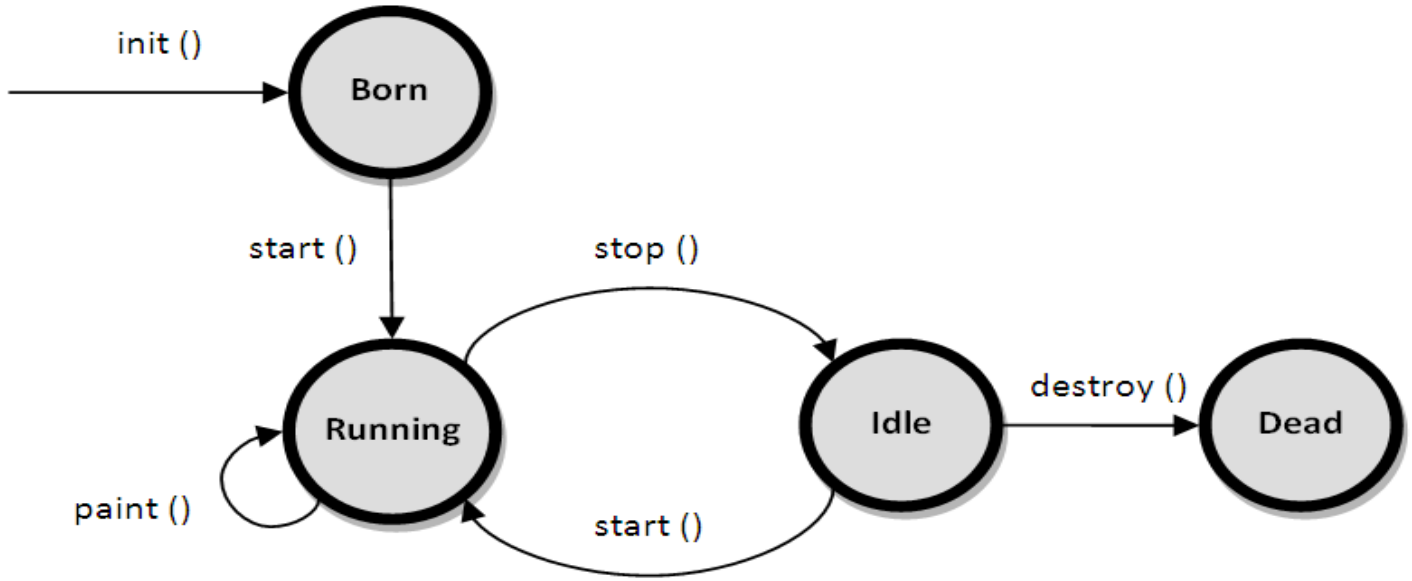
Applet Lifecycle Methods

- **stop():** The stop() method stops the execution of the applet.
- The stop() method executes when the applet is minimized or when moving from one tab to another in the browser.



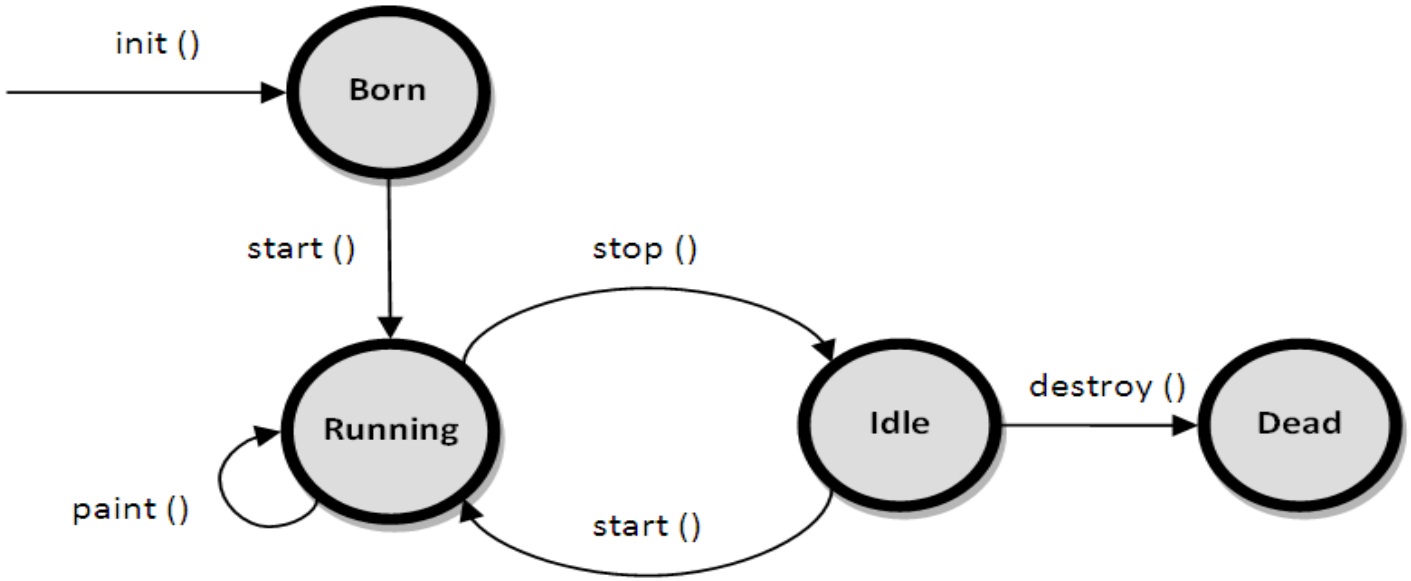
Applet Lifecycle Methods

- **destroy():** The destroy() method executes when the applet window is closed or when the tab containing the webpage is closed.
- *stop()* method executes just before when destroy() method is invoked. The destroy() method removes the applet object from memory.



Applet Lifecycle Methods

- **paint():** The paint() method is used to redraw the output on the applet display area.
- The paint() method executes after the execution of *start()* method and whenever the applet or browser is resized.



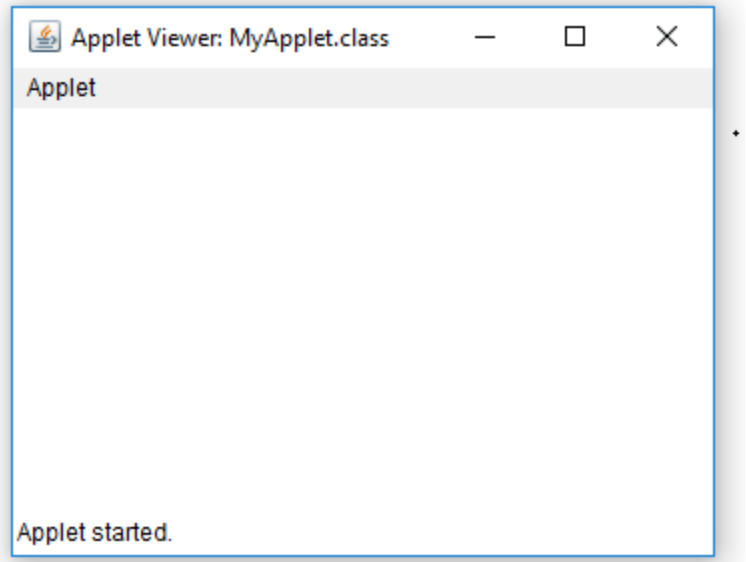
Applet Lifecycle

- The method execution sequence when an applet is executed is:
- init()
- start()
- paint()
- The method execution sequence when an applet is closed is:
- stop()
- destroy()

Applet Lifecycle Program

- // Program for demonstrating java applet life cycle

```
// Program for demonstrating java applet life cycle
import java.awt.*;
import java.applet.*;
public class MyApplet extends Applet
{
        public void init()   {
                System.out.println("Applet initialized");
        }
        public void start()             {
                System.out.println("Applet execution started");
        }
        public void stop()              {
                System.out.println("Applet execution stopped");
        }
        public void paint(Graphics g)           {
                System.out.println("Painting...");
        }
        public void destroy()           {
                System.out.println("Applet destroyed");
        }
}
```



Output
run:
Applet initialized
Applet execution started
Painting...
Applet execution stopped
Applet destroyed
BUILD SUCCESSFUL (total time: 16 seconds)

Restrictions imposed on Java applets

- Due to security reasons , the following restrictions are imposed on Java applets:
  1. An applet cannot load libraries or define native methods.
  2. An applet cannot ordinarily read or write files on the execution host.
  3. An applet cannot read certain system properties.
  4. An applet cannot make network connections except to the host that it came from.
  5. An applet cannot start any program on the host that’s executing it.

Difference between Applet and application

- There are some important differences between an applet and a standalone Java application, including the following −

1. An applet is a Java class that extends the java.applet.Applet class.
2. A main() method is not invoked on an applet, and an applet class will not define main().
3. Applets are designed to be embedded within an HTML page.
4. When a user views an HTML page that contains an applet, the code for the applet is downloaded to the user's machine.

Difference between Applet and application

5. A JVM is required to view an applet.
6. The JVM can be either a plug-in of the Web browser or a separate runtime environment.
7. The JVM on the user's machine creates an instance of the applet class and invokes various methods during the applet's lifetime.
8. Applets have strict security rules that are enforced by the Web browser.
9. The security of an applet is often referred to as sandbox security, comparing the applet to a child playing in a sandbox with various rules that must be followed.
10. Other classes that the applet needs can be downloaded in a single Java Archive (JAR) file.

Applet Architecture

- Applets are event driven.
- An applet waits until an event occurs.
- The run-time system notifies the applet about an event by calling an event handler that has been provided by the applet.
- Once this happens, the applet must take appropriate action and then quickly return control to the system.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
public class Q2 extends Applet
            implements ActionListener
{
   TextField t1 = new TextField(10);
   TextField t2 = new TextField(10);
   TextField t3 = new TextField(10);
   Label l1 = new Label("FIRST NO=:");
   Label l2 = new Label("SECOND NO:");
   Label l3 = new Label("SUM:");
   Button b = new Button("ADD");
 public void init()
   {    t1.setForeground(Color.Red);
      add(l1);   add(t1);        add(l2);   add(t2);
      add(l3);   add(t3);        add(b);
      b.addActionListener(this);
   }
public void actionPerformed(ActionEvent e)
   {
      if (e.getSource() == b)
      {
         int n1 = Integer.parseInt(t1.getText());
         int n2 = Integer.parseInt(t2.getText());
         t3.setText(" " + (n1 + n2));
      }
   }
}
```

Applet Architecture (Event handling)

```
// Applet for addition of two nymbers
package AppletPrograms;
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
public class AdditionApplet extends Applet
                implements ActionListener
{
   TextField t1 = new TextField(10);
   TextField t2 = new TextField(10);
   TextField t3 = new TextField(10);
   Label l1 = new Label("FIRST NO=:");
   Label l2 = new Label("SECOND NO:");
   Label l3 = new Label("SUM:");
   Button b = new Button("ADD");
   public void init()
   {
      t1.setForeground(Color.red);
      add(l1);    add(t1);
      add(l2);    add(t2);
      add(l3);    add(t3);
      add(b);
      b.addActionListener(this);
   }
   public void actionPerformed(ActionEvent e)
   {
      if (e.getSource() == b)
      {
         int n1 = Integer.parseInt(t1.getText());
         int n2 = Integer.parseInt(t2.getText());
         t3.setText(" " + (n1 + n2));
      }
   }
}
```

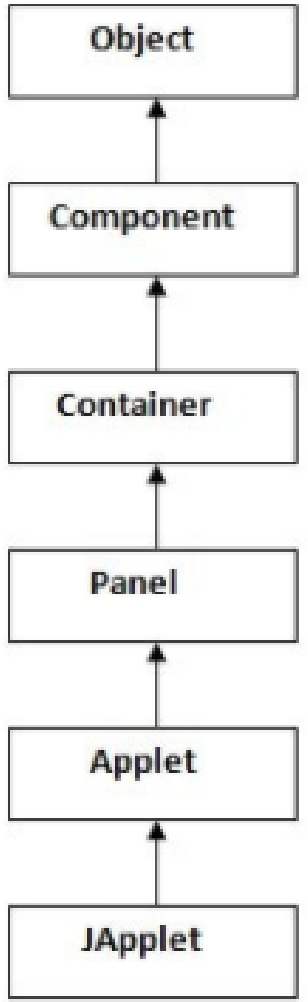
Module 1- Section 3

3.4 Applet Class

Applet Class

- When we create our own applet classes, they descend from the Applet class and inherit methods and variables defined in the Applet class.
- Not only do our own applets inherit from the Applet class but also from the super classes of Applet such as Panel, Container, Component and Object.

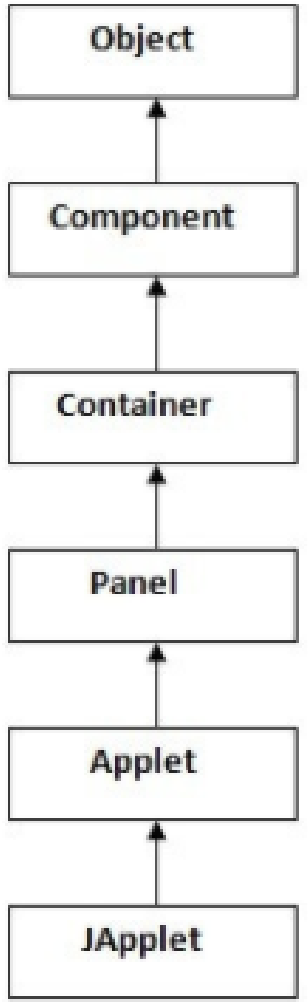
Hierarchy of Applet



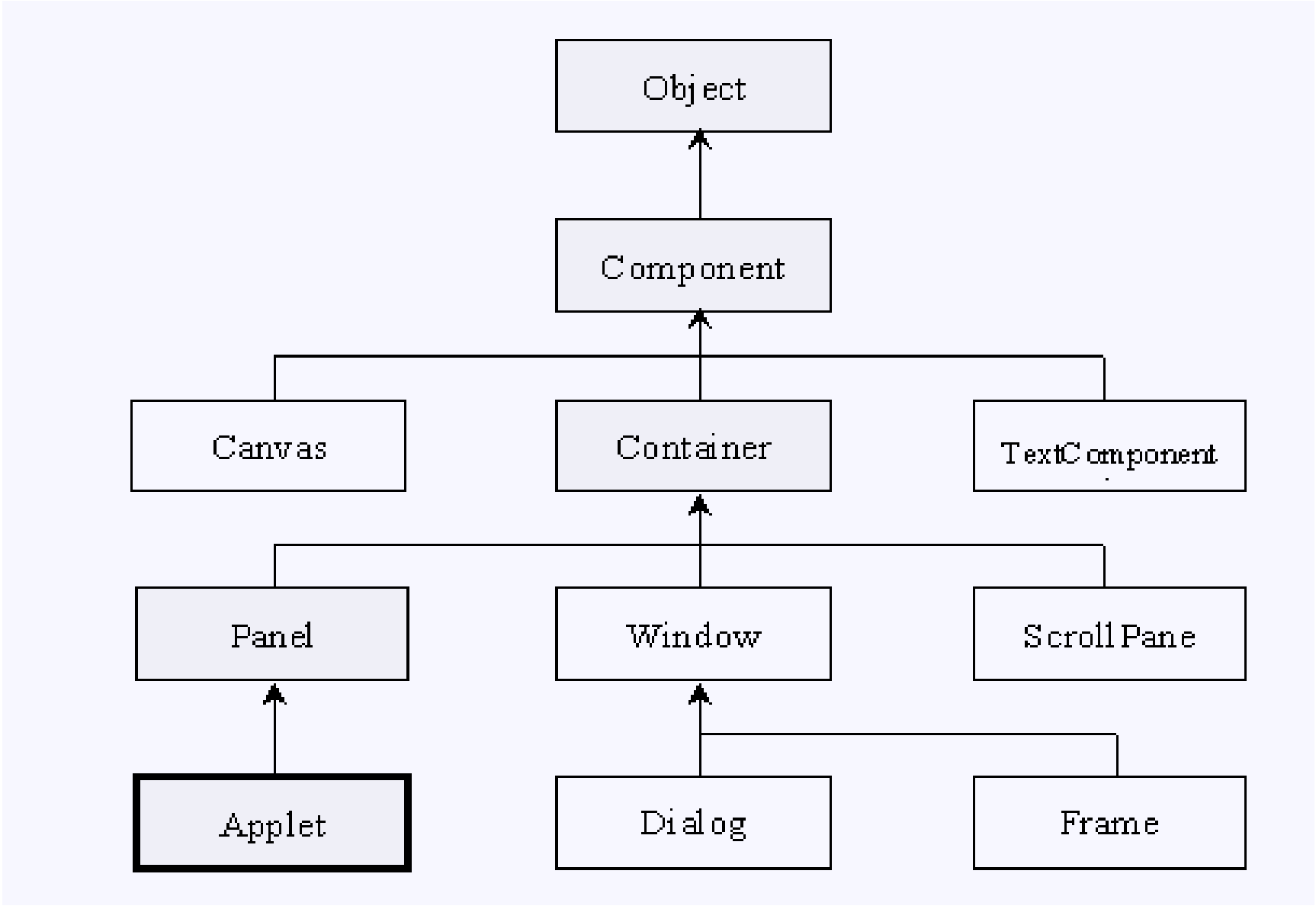
Applet Class

- We are also allowed to re-define an inherited method by overriding that method.
- The diagram below shows the class hierarchy, highlighting the super classes of Applet .

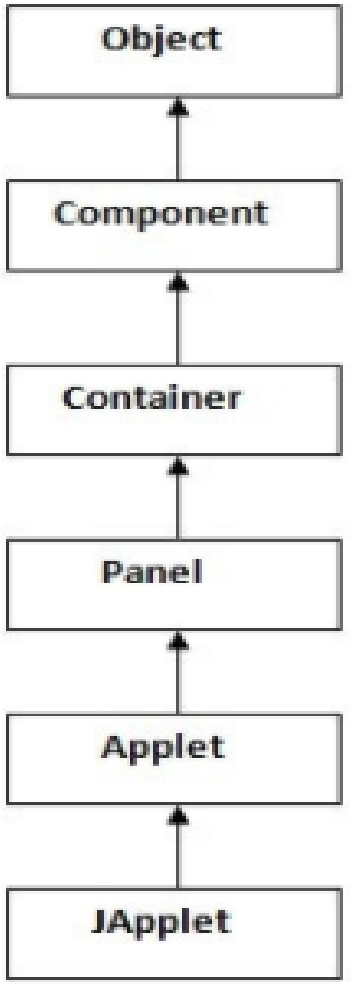
Hierarchy of Applet



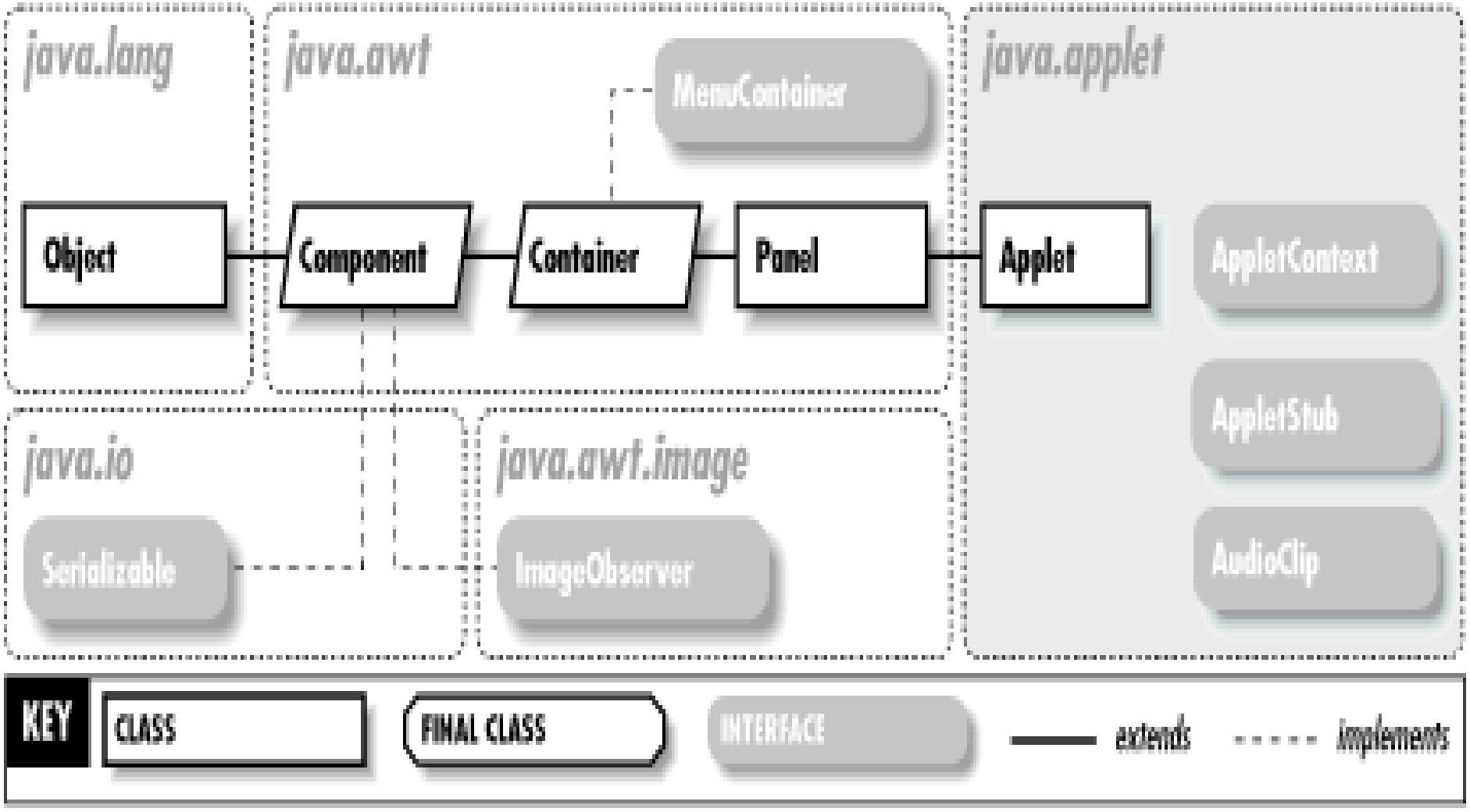
Applet Class : Detail Diagrams

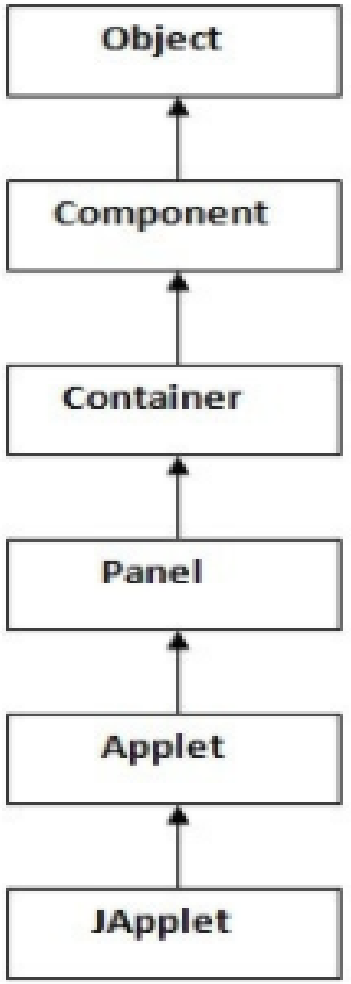


Hierarchy of Applet



Hierarchy of Applet





Applet Class

- The Applet class extends the AWT Panel class, which extends the AWT Container class, which extends the AWT Component class.

1. **From Component,** an applet inherits the ability to draw components and handle events.
2. **From Container,** an applet inherits the ability to include other components and to have a layout manager to control the size and position of those components.
3. **From Applet,** an applet inherits several capabilities, such as loading and unloading.

Applet Class

- Although the Applet class inherits many features from other classes such as Panel, Container, Component and Object there are **five** main methods of the Applet class. These are:

1. init() - An applet can initialize itself.
2. start() - An applet can start running.
3. paint() - Graphics shapes such as text, circles rectangles etc. may be drawn onto the applet.
4. stop() - An applet can stop running.
5. destroy() - An applet can perform a final cleanup, in preparation for being unloaded and destroyed.

The Applet Class Methods

- Applet class provides several useful methods to give you full control over the execution of an applet. Some important methods are:

1. void init()
2. void start()
3. void stop()
4. void destoy()
5. String getParameter(String ParameterName)
6. Image getImage(URL url)
7. void play(URL url)
8. showStatus(String str)

Methods	Description
void init()	The first method to be called when an applet begins its execution.
void start()	Called automatically after init() method to start the execution of an applet.
void stop()	Called when an applet has to pause/stop its execution.
void destoy()	Called when an applet is finally terminated.
String getParameter (String ParameterName)	Returns the value of a parameter defined in an applet tag in the html code
Public void paint(Graphics g)	It draw the output over the applet. It call various methods of Graphics class.
void play(URL url)	Plays an audio clip found at the specified at location, url
showStatus(String str)	Shows a message in the status window of the browser or appletviewer.

Module 1- Section 3

3.5 The HTML Applet Tag

Applet Tag

- The HTML <applet> tag specifies an applet.
- It is used for embedding a Java applet within an HTML document.

- <applet
  - code="HelloWorld.class"
  - width=200
  - height=60
  - >
- </applet>

Applet Tag

- The **<applet>** tag in HTML was used to *embed Java applets into any HTML document*.
- The **<applet>** tag was deprecated in HTML 4.01, and it's support has been completely discontinued starting from HTML 5.
- Alternatives available in HTML 5 are the **<embed>** and the **<object>** tags.

Applet Tag Attributes

- The <applet> tag takes a number of attributes, with one of the most important being the **code** attribute.
- This **code** attribute is used to link a Java applet to the concerned HTML document.
- It specifies the file name of the Java applet.

Applet Tag Attributes

- **Syntax:**
- <applet *attribute1 attribute2....*>
- </applet>

Applet Tag Attributes : Example

```
<html>
   <applet
     code="HelloWorld"
     width=200
     height=60>
   </applet>
</html>
```

- Here, HelloWorld is the class file, which contains the applet. It is specified by **code** attribute.
- The **width** and **height** attributes determine the width and height of the applet in pixels when it is opened in the browser.

Applet Tag Attributes : Example

- The important attributes supported by applet tag are given in Table.

Attribute	Value	Description
align	URL	Deprecated – Defines the text alignment around the applet
alt	URL	Alternate text to be displayed in case browser does not support applet
archive	URL	Applet path when it is stored in a Java Archive ie. jar file
code	URL	A URL that points to the class of the applet
codebase	URL	Indicates the base URL of the applet if the code attribute is relative
height	pixels	Height to display the applet

Attribute	Value	Description
hspace	pixels	*Deprecated* − Defines the left and right spacing around the applet
name	name	Defines a unique name for the applet
object	name	Specifies the resource that contains a serialized representation of the applet's state.
title	test	Additional information to be displayed in tool tip of the mouse
vspace	pixels	*Deprecated* − Amount of white space to be inserted above and below the object.
width	pixels	Width to display the applet.

Module 1- Section 3

Graphics Class and its various Methods in an Applet

Graphics class in Java

- java.awt.Graphics class provides many methods for graphics programming.
- It is available in java.awt package and can be imported as
- Import **java.awt.Graphics**

Important Methods of Graphics class

1. drawstring()
2. drawRect ()
3. fillRect ()
4. drawOval ()
5. fillOval ()
6. drawLine ()
7. drawArc ()
8. fillArc ()
9. setColor ()
10. setFont ()

Important Methods of Graphics class

public abstract void **drawstring** (String str, int x, int y): is used to draw the specified string.

public void **drawRect** (int x, int y, int width, int height): draws a rectangle with the specified width and height.

public abstract void **fillRect** (int x, int y, int width, int height): is used to fill rectangle with the default color and specified width and height.

public abstract void **drawOval** (int x, int y, int width, int height): is used to draw oval with the specified width and height.

public abstract void **fillOval** (int x, int y, int width, int height): is used to fill oval with the default color and specified width and height.

Important Methods of Graphics class

- public abstract void **drawLine** (int x1, int y1, int x2, int y2): is used to draw line between the points(x1, y1) and (x2, y2).
- public abstract boolean **drawImage** (Image img, int x, int y, ImageObserver observer): is used draw the specified image.
- public abstract void **drawArc** (int x, int y, int width, int height, int startAngle, int arcAngle): is used draw a circular or elliptical arc.
- public abstract void **fillArc** (int x, int y, int width, int height, int startAngle, int arcAngle): is used to fill a circular or elliptical arc.
- public abstract void **setColor** (Color c): is used to set the graphics current color to the specified color.
- public abstract void **setFont** (Font font): is used to set the graphics current font to the specified font.